Project: Super Mario
Finals

DANG-KHANG NGUYEN, 25125017, ndkhang2535@apcs.fitus.edu.vn
DUC-HUY NGUYEN, 25125014, ndhuy2526@apcs.fitus.edu.vn
MINH-NHAT LE, 25125032, bvthanh2542@apcs.fitus.edu.vn
VIET-THANH BUI, 25125065, bvthanh2542@apcs.fitus.edu.vn
MAI-THUY VU, 23125046

02^{nd} of September, 2026

Abstract

Object-Oriented Programming (OOP) is a foundational paradigm in Computer Science for building robust, modular software systems. Developing a 2D platformer like *Super Mario Bros.* provides an ideal framework for applying key OOP principles—such as inheritance, encapsulation, polymorphism, and abstraction—alongside foundational software design patterns. This project introduces a 2D game engine built in C++ using the SFML library, featuring a deterministic fixed-timestep game loop, stack-based state management, dynamic custom map parsing, and responsive character physics.

Purpose

This project provides a modular, extensible 2D platformer clone of *Super Mario Bros.* built in C++ using SFML, designed to demonstrate core Object-Oriented Programming principles and software design patterns in game architecture.

Key features

The platformer incorporates a comprehensive suite of systems and gameplay mechanics:

- **Player Mechanics & Characters**: Supports multiple selectable characters—Mario and Luigi—with custom movement, jumping physics, and state transformations including Small, Super, Fire, and Star states.
- **Diverse Enemy AI**: Features distinct enemy behaviors for Goombas, Koopa Troopas, Koopa Paratroopas, Hammer Bros, Lakitus, Spinies, and Bowser.
- **Custom Map Parser**: Loads multi-area levels from custom text-based `.map` files supporting solid terrain, interactive blocks, warp pipes, moving platforms, and decorative scenery.
- **Engine & Game State System**: Built on a fixed-timestep game loop and a stack-based state manager transitioning seamlessly between Menu, Play, and Game Over states.
- **Design Pattern Integration**: Implements core software design patterns including State, Singleton, and Factory to achieve high modularity and maintainability.

Mission

This project aims to demonstrate how clean Object-Oriented design, structured engine architecture, and software design patterns can be applied to solve complex interactive problems in 2D game development.

Contents

1 Introduction

This report documents the design, architecture, and technical implementation of a 2D *Super Mario Bros.* platformer game developed in C++ using the SFML multimedia library. Developed as a final project for the Object-Oriented Programming (OOP) course, the project demonstrates the practical application of core OOP principles—inheritance, encapsulation, polymorphism, and abstraction—alongside software design patterns including State, Singleton, Factory, etc.

Overview of the Report

The purpose of this document is to serve as both a technical architectural reference for software engineers and a practical operational guide for players and level authors. The report covers the software stack from high-level engine management down to specific collision-handling mechanisms and enemy AI routines.

Report Structure & Roadmap

To help readers navigate through specific topics, this document is organized into the following sections:

- **Chapter 1 — Introduction**: Contains the abstract, purpose, key features, mission statement, and structural guide for this report.
- **Chapter 2 — Project Architecture**: Outlines the Model-View-Controller (MVC) division, structural class diagrams, engine scaffolding (`AppEngine`, `StateManager`), global tracking (`GameManager`), and entity hierarchies.
- **Chapter 3 — Implementation Details**: Explains the technical execution of key mechanics, including the fixed-timestep loop, custom `.map` file parsing, tile/entity collision resolution, state transformations, and enemy AI routines.
- **Chapter 4 — Development Cycle**: Describes the development workflow, milestone phases, and technical notes accumulated during development.
- **Chapter 5 — Tools Used**: Outlines the development environments, CMake build pipeline, SFML framework, version control, and media asset sources.
- **Chapter 6 — Features**: Details mandatory gameplay capabilities (3-level progress, sound effects, AI) and advanced features (such as character selection between Mario and Luigi).
- **Chapter 7 — Installation**: Provides instructions for setting up dependencies, configuring CMake, compiling the source code, and launching the executable.
- **Chapter 8 — Usage**: Serves as a manual covering player controls, power-up state transitions, warp pipe navigation, and the symbol reference for authoring custom map files.
- **Chapter 9 — Conclusion**: Summarizes project outcomes, known technical limitations, and prospective future enhancements.
- **Chapter 10 — References**: Lists core academic textbooks, API documentation, and technical reference wikis used.

Finding Topics of Interest

Readers seeking specific subsystems or implementation recipes can navigate directly to these key areas:

- **Game Loop & State Management**: See *Program loop* for details on `AppEngine`, the fixed 60 FPS update step, and the deferred `StateManager` stack.
- **Player Characters & Power-Up States**: See Game entities and Mechanics for the `PlayerState` pattern (Small, Super, Fire, Star) and character movement properties.
- **Enemy AI & Spawning Rules**: See `EnemyFactory` rules and individual enemy behavior specs (Goomba, Koopa, Hammer Bro, Lakitu, Bowser).
- **Level Design & Custom Map Authoring**: See `.map` file specification, grid coordinates, and symbol binding table.

Disclaimer

The report might be slightly outdated since parts of it were written before the final submission of the project.

2 Project architecture

The software architecture of this *Super Mario Bros.* implementation is structured around the Model-View-Controller (MVC) design pattern to maintain a clear separation of concerns, high modularity, and strict adherence to Object-Oriented Programming principles. The system decouples runtime control logic (`controller::`), data models (`model::`), and visual rendering routines (`view::`) into distinct architectural layers. The following subsections outline the overall architectural blueprint, ranging from the core execution loop and global state managers to structural level representations and dynamic game entities.

2.1 Program loop

The program loop serves as the core execution engine of the application, orchestrating window management, input polling, fixed-timestep physics updates, and frame rendering. Utilizing the State design pattern, it manages top-level application screens and transparent overlays through a stack-based manager that defers state transitions to guarantee execution safety.

2.1.1 AppEngine

The `AppEngine` class, situated within the `controller::` namespace, serves as the primary driver for the application execution cycle. It owns both the single `sf::RenderWindow` instance and the top-level `StateManager`. The primary entry point, `AppEngine::run()`, executes a fixed-timestep game loop designed to keep physics calculations and game logic deterministic regardless of rendering frame rates.

Each iteration of the main loop follows a strict four-phase sequence:

1. **Input processing**: Polling window events and delegating them through `processInput()` to active states.
2. **Fixed-step update**: Consuming accumulated elapsed time in discrete `TimeStep` slices (1/60 s) via `update(TimeStep)`.
3. **Rendering**: Clearing the window, drawing visible states via `render()`, and displaying the window content.
4. **Deferred state transitions**: The program executes queued state stack operations through `states.applyPending()`.

To protect against frame stalls or time-step accumulation spikes during window resizing or dragging, the loop caps the maximum frame delta time using a `MaxFrameTime` constant (0.25 s). The main loop runs continuously until the render window is closed or the state stack becomes empty.

2.1.2 StateManager

The `StateManager` class manages application screens using an internal stack of states stored as `std::vector<std::unique_ptr>`. It exposes methods for controlling stack composition—including `pushState()`, `popState()`, `replaceState()`, `clear()`, and `empty()`.

The manager coordinates execution flow based on stack state and screen visibility:

- **Input & Update Routing**: Input event handling (`handleEvent`) and fixed-step logic updates (`update`) are dispatched strictly to the active state at the top of the stack.
- **Multi-layer Rendering**: The `render` method traverses the stack from the lowest non-hidden state up to the top. This enables transparent overlay states (such as pause screens) to render over visible gameplay screens underneath.
- **Deferred Stack Operations**: State modifications requested during execution are not applied immediately. Instead, requests are queued internally and enacted when `applyPending()` is invoked at the end of the frame cycle. This prevents container invalidation and premature object destruction when a state requests a transition during its own update call.

2.1.3 GameState hierarchy

The `GameState` class defines an abstract base interface implementing the State design pattern. Every individual application screen derives from `GameState` and implements its core lifecycle methods:

- `onEnter()` and `onExit()`: Lifecycle callbacks executed when a state is pushed onto or popped from the stack.
- `handleEvent(const sf::Event& event)`: Pure virtual method for processing user keyboard and window input events.
- `update(float deltaTime)`: Pure virtual method for executing fixed-step screen logic.
- `render(sf::RenderWindow& window)`: Pure virtual method handling drawing calls to the window.
- `isTransparent()`: Returns a boolean (defaulting to `false`) indicating whether underlying states in the stack should be rendered.

Each `GameState` maintains an internal `StateManager* manager` back-pointer assigned automatically upon being pushed. This allows concrete states to trigger screen transitions directly. The core engine includes three concrete state implementations:

- `MenuState`: Controls the title screen. It invokes `GameManager::reset()` upon entry to reinitialize score and life counters, transitioning to gameplay upon user input.
- `PlayState`: Manages active gameplay, serving as the integration container for the `TileMap`, `TileMapRenderer`, game world, and player entity.
- `GameOverState`: Displays final performance statistics upon player defeat before allowing return transitions to the main menu.

2.2 Global structures

Global structures manage cross-state data persistence throughout the entire session lifecycle. By centralizing key runtime metrics—such as player score, remaining lives, and active level indicators—these structures allow decoupled game states to query and modify overall progress seamlessly.

2.2.1 GameManager

The `GameManager` class, located in the `model::` namespace, is implemented as a Singleton design pattern to provide centralized access to persistent game session state. It guarantees the existence of a single, globally accessible instance via the static `instance()` method, while its copy constructor and copy assignment operator are explicitly deleted to prevent duplicate instances.

`GameManager` acts as the global data hub across state transitions, tracking three primary metrics throughout a game session:

- **Score**: Maintained via an internal integer counter and accessed/updated through `getScore()` and `addScore(int)`.
- **Lives**: Monitored via an internal counter accessed through `getLives()`, incremented via `addLife()`, and decremented using `loseLife()` (which is floored at 0).
- **Level Progress**: Managed using `getCurrentLevel()`, `setCurrentLevel(int)`, as well as `nextLevel()` to track the active stage.

The class exposes default configuration constants, including `StartingLives` (3) and `FirstLevel` (1). When `MenuState` is entered, it calls `reset()` to restore all global metrics back to these initial values. During gameplay, `PlayState` updates scores or life counts based on entity interactions and queries `isGameOver()`—which returns `true` whenever lives reach zero—to determine when to trigger a state transition to `GameOverState`.

2.2.2 Vector2 and CollisionResult

The `Vector2` structure provides a lightweight two-dimensional vector primitive containing floating-point `x` and `y` attributes. It serves as the primary data structure for tracking spatial coordinates, bounding box sizes, and movement velocity vectors across the `Entity` and `Character` hierarchies.

The `CollisionResult` structure serves as a feedback container that encapsulates the outcome of spatial queries between entities or environment geometry. It holds a boolean `collided` flag indicating whether an intersection occurred, a `CollisionType` classification denoting the direction or surface of contact, and a pointer to the target `Entity` involved in the collision.

2.2.3 TileMapRenderer & AssetManager

The `TileMapRenderer` class in the `view::` layer handles drawing environmental tile maps onto an `sf::RenderWindow` instance. It stores a `tilesetTexture` atlas and maintains an internal `unordered_map<char, IntRect>` mapping ASCII map symbols to sub-rectangles within the texture atlas. Using `registerTile(symbol, col, row)`, symbols are bound to atlas regions, enabling `render(window, map)` to iterate over active map grids and draw terrain elements. Certain interactive elements—such as coin blocks (`C`) and brick blocks ()—are excluded from static tile atlas rendering and are instead drawn and updated dynamically as individual entities.

The `AssetManager` class acts as a centralized resource hub that loads, stores, and supplies shared textures and font files required across the application. It allows states like `MenuState` and `PlayState` to query required graphics and typography on demand, avoiding redundant asset loading from disk.

2.2.4 TileMap and Map Loader

The `TileMap` class stores the spatial layout of an area using a 2D character grid (`vector<vector> tiles`). It exposes functions such as `getTile(row, col)`, `getRows()`, and `getColumns()` to query tile data, maintaining standard structural dimensions including `Rows` (16 per area) along with tile width and height parameters.

Map data is ingested via text-based `.map` files using two main loading routines:

- `Level::loadFromFile`: The multi-area loader used during active gameplay, capable of handling multi-area transitions and header declarations.
- `TileMap::loadFromFile`: A single-area legacy loader path where all header lines precede grid rows.

The parser reads grid lines in a bottom-up order, where row 0 corresponds to the bottom row of the screen ($y = (15 - \text{row}) \times 16$). Headers prefixed with `;` supply metadata and bindings:

- **Level and Area Headers**: `; name=` sets display names for the HUD, `; next=` specifies the subsequent map file, `; area` defines area boundaries, and `; world=` selects palette and physics themes (`overworld`, `underground`, `underwater`, `castle`).
- **Binding Tokens**: `; pipe=` establishes warp destination columns and areas, while `; slider=` configures movement axis, distance, and speed for moving platforms.

During line parsing, the loader classifies ASCII symbols into solid terrain (`G`, `s`, `c`, `r`, pipe cells `P`/`Q`/`p`/`q`), dynamic items and triggers (`M`, `C`, `#`, `B`, `$`, `E`, `=`, `X`), enemy spawn digits (`0`–`8`), and decorative background scenery (`O`, `T`, `w`, `m`, `k`, `v`, `x`, `A`, `H`).

2.2.5 World and Level

The `Level` structure manages multi-area stage flow and level transitions. It encapsulates one or more `TileMap` areas, coordinating area switching when the player uses warp pipes specified by `; pipe=` tokens or triggers stage completion via a level goal (`E`).

The `World` class serves as the runtime container housing active entities and managing environmental physics. During each fixed update cycle in `PlayState`, `World::step(deltaTime)` executes physics calculations and resolves collisions across all contained entities.

`World` provides key interfaces for entity interactions:

- `World::getPlayer()`: Allows entities (such as Lakitu) to query player coordinates for targeted tracking and movement.
- `World::spawn(Entity)`: Handles runtime entity creation. While level map files place standard enemies via map digits, dynamic consequences—such as thrown hammers, Spiny Eggs, and hatched Spinies—are instantiated at runtime through `World::spawn`.

2.2.6 Entity and Character base classes

The `Entity` class serves as the abstract base class for all spatial objects within the game world. It encapsulates positional coordinates and dimensions via `Vector2` structures (`#position` and `#size`), providing accessor and mutator methods alongside virtual `update(dt)` routines. Concrete spatial objects within the game environment—including dynamic entities and interactive terrain blocks—derive from `Entity`.

The `Character` class extends `Entity` to form the base class for all mobile actors, including player characters, enemies, and non-player characters. It introduces motion, state, and collision properties:

- **Movement & State Attributes**: Tracks movement velocity (`#velocity`), spatial orientation (`#direction`, `#facingRight`), health counters (`#health`), vitality status (`#alive`), and animation states (`#animState`).
- **Environmental Interaction**: Maintains a reference to the active map (`#mapPtr`) to execute tile collision checks (`resolveTileCollisions()`) and ground detection (`isOnGround()`).
- **Physics & Lifecycle Management**: Provides routines for applying gravity (`applyGravity(dt)`), velocity clamping (`clampVelocity()`), taking damage (`takeDamage(amount)`), and handling entity collision events (`onCollision(other)`).

2.2.7 Player, Mario, and Luigi

The `Player` class derives from `Character` and represents the user-controlled character. It manages gameplay statistics—such as score counters (`#score`), coin counts (`#coins`), remaining lives (`#lives`), and invulnerability timers (`#damageCooldown`)—while managing active state transformations through a `std::unique_ptr` container. It implements input handling via `handleInput()` and exposes helper methods to grant score (`addScore`), award coins (`addCoin`), grant lives (`addLife`), and trigger power-up transformations (`becomeSuper()`, `becomeFire()`, `becomeStar()`).

The engine provides two distinct playable character classes derived from `Player`:

- `Mario`: Configured with balanced mobility parameters, featuring a walking speed of 180, a running speed of 320, and a jump force of -450.
- `Luigi`: Configured with specialized physical traits, featuring a lower walking speed of 160 and running speed of 280, but a higher jump force of -540 allowing him to reach higher platforms.

2.2.8 PlayerState pattern

Player power-up states and visual/functional transformations are managed using the State design pattern via the `PlayerState` abstract interface. `PlayerState` defines lifecycle hooks (`onEnter`, `onExit`), frame update routines (`update`), state name queries (`getStateName`), animation state queries (`getAnimState`), and damage resolution methods (`takeDamage`).

The architecture implements four concrete player states:

- `SmallState`: Represents default player form. Sustaining damage while in this state results in loss of life.
- `SuperState`: Formed when collecting a Mushroom. Sustaining damage demotes the player back to `SmallState` rather than causing instant death.
- `FireState`: Formed when collecting a Fire Flower. Enables projectile capabilities tracked via an internal `fireCooldown` timer, exposing `canShoot()` and `shoot()` methods to emit fireballs. Sustaining damage demotes the player back to `SuperState`.
- `StarState`: Formed when collecting a Star. Implements a decorator pattern that wraps around the player's `previousState` and enforces temporary invincibility over a fixed `duration`. It monitors expiration via `checkExpiration()` and `getRemainingTime()`, returning ownership back to the `previousState` once the timer elapses.

2.2.9 NPCs and Princess

Non-player characters are derived from the `NPC` class, which extends `Character` to provide non-combat interaction support. `NPC` maintains dialog strings (`#dialogue`) and interaction flags (`#interactable`), exposing an abstract `interact(player)` interface alongside dialog accessor methods.

Concrete NPC implementations include:

- `Princess`: Derived from `NPC`, implementing custom interaction logic triggered when reached by the player.
- `MushroomRetainer`: Represents Toad, derived from `NPC` to handle endgame interactions. In level map files, Toad is placed using the `R` symbol as a 16×24 non-solid backdrop entity.

2.2.10 EnemyFactory and Enemy AI

Enemy creation follows a strict map-driven factory rule: no enemy placement is hardcoded in C++; instead, level authors place map digits (`0`–`8`) that are instantiated dynamically by `EnemyFactory`. The only exception is runtime projectile emission (such as thrown hammers or Spiny Eggs), which occurs via `World::spawn`.

The enemy roster incorporates distinct behavioral AI patterns:

- `0` **Goomba**: Walks in a fixed direction at constant speed (40), reverses direction upon hitting walls, and falls off ledges. Squashed into a flat sprite when stomped.
- `1` **Koopa Troopa**: Walks like a Goomba. Stomping retracts it into a shell state; stationary shells are harmless until kicked, after which they slide at high speed (250), knocking out enemies and ricocheting off walls.
- `2` **Koopa Paratroopa**: Winged variant that executes a hopping walk pattern by combining standard walking speed (40) with a fixed bounce upon landing. Stomping removes its wings to produce a standard walking Koopa.
- `3` **Hammer Bro**: Patrols a short beat around its spawn point, hopping frequently while aiming and throwing hammers towards player coordinates on a fixed cooldown (2.0 s). Thrown hammers travel in an arc and ignore terrain collision.
- `4` **Lakitu**: Flies overhead without gravity or tile collision, using `World::getPlayer()` to track player horizontal movement and dropping Spiny Eggs on a fixed timer (3.0 s).
- `5` **Spiny**: Non-stompable enemy covered in spikes. Landings on a Spiny deal damage to the player. Hatches from thrown Spiny Egg projectiles upon ground contact.
- `7` **Bowser**: Boss entity spanning two tiles in height. Paces along a short path, jumps periodically, and breathes fireballs on a 2.5 s cooldown. Non-stompable, featuring a 5-hit health pool.
- `6` **Cheep Cheep** & `8` **Piranha Plant**: Cheep Cheeps swim horizontally across rows, while Piranha Plants emerge from pipe mouths on a fixed timer.

2.2.11 Blocks, Items, and Interactive mechanisms

Interactive terrain and items inherit from `Block` (derived from `Entity`), which tracks tile symbols (`#tileSymbol`) and solidity flags (`#solid`).

Key block and item implementations include:

- `CoinBlock` (`C`): Derived from `Block`, tracking coin availability via `#coinAvailable`. When struck from below, it dispenses a coin, Mushroom, Fire Flower, or Star before transforming into a used block.
- **Brick Block** (`# B`): Bounces when bumped by Small Mario, but shatters into animated fragments when struck from below by Super Mario.
- **Free-Standing Coin** (`$`) & **Retainer** (`R`): Non-solid collectibles and static NPCs collected or interacted with upon contact.

Specialized interactive level triggers direct environment mechanics:

- **Level Goal** (`E`): A 4-tile-tall non-solid trigger zone that fires end-of-level routines when touched by the player.
- **Chain Trigger** (`X`): A non-solid trigger (the axe) that erases all bridge deck cells (`c`) within an area upon contact, dropping structures into pits or lava.
- **Moving Platforms Sliders** (`==`): Platforms defined by 2-cell grid runs, bound to motion direction, distance, and speed parameters via `; slider=` tokens.
- **Firebars** (`r`) & **Lava Bubbles** (`b`): Firebars spawn a rotating line of 4 flame elements sweeping from a central block mount, while lava bubbles leap vertically out of lava crest rows (`v`).

3 Implementation

3.1 Overview

Below is a (simplified) class diagram of the project:

4 Development Cycle

The development of the *Super Mario Bros.* engine followed a structured, phase-driven engineering workflow aimed at establishing a stable architectural scaffold before adding complex gameplay mechanics. By dividing engine responsibilities across modular subsystems—ranging from core window management to physics resolution and map parsing—the project maintained a clean Model-View-Controller (MVC) separation throughout the development lifecycle.

4.1 Workflow

The project utilized a CMake-based compilation pipeline structured around automated file scanning and explicit layer separation:

- **Build System Configuration**: The project build system relies on CMake's macro `file(GLOB_RECURSE ...)` to automatically discover source files located within the `src` and `include` directories.
- **Reconfiguration Requirements**: Because CMake scans source trees during its initial configuration phase, adding or removing `.cpp` or `.h` files requires executing a full re-scan command (`cmake -S . -B build`) prior to compiling (`cmake -build build`).
- **Modular Issue Tracking**: Engine construction was organized into five core technical issues:
  1. *Issue 1*: Engine Core, Game Loop, and State Manager.
  2. *Issue 2*: Rendering Pipeline and Asset Management.
  3. *Issue 3*: Physics Engine and Collision Detection.
  4. *Issue 4*: Level Parsing and Map Loading.
  5. *Issue 5*: Player Control and Mechanics.
- **Dependency Management**: Executing the resulting binary requires placing SFML dynamic link libraries alongside MinGW binaries on the system `PATH`.

4.2 Project phases

Development progressed through five sequential phases to ensure engine stability and incremental feature verification:

- **Phase 1 — Core Engine Scaffold**: Constructed the `AppEngine` class, established the fixed 1/60 s update loop, implemented the stack-based `StateManager`, and integrated placeholder states (`MenuState`, `PlayState`, `GameOverState`) alongside the global singleton class `GameManager`.
- **Phase 2 — Asset Management & Visual Rendering**: Built the `AssetManager` for centralized resource management and implemented `TileMapRenderer` to map ASCII characters to specific texture atlas sub-rectangles.
- **Phase 3 — Physics & Collision Resolution**: Integrated deterministic broad-phase and narrow-phase collision handling between mobile actors (`Character`) and solid environment tiles (`TileMap`).
- **Phase 4 — Map Parsing & Level Architecture**: Created `Level` and `TileMap` loaders capable of parsing custom `.map` text files, computing bottom-up row offsets ($y = (15 - \text{row}) \times 16$), binding area parameters (`; world=`), and linking warp portals (`; pipe=`) and sliders (`; slider=`).
- **Phase 5 — Gameplay Mechanics, Entities & AI**: Implemented the `PlayerState` pattern (Small, Super, Fire, Star), added playable characters (Mario and Luigi), constructed `EnemyFactory` for digit-based enemy spawning (`0`–`8`), and realized interactive block mechanics.

4.3 Comments

Several design choices, technical adaptations, and temporary development stubs were incorporated during the engineering cycle:

- **Deliberate Design Deviations**:
  - *Color Variants*: Omitted red and green Koopa variants to standardize patrol behavior, halving required art assets and state complexity.
  - *Lakitu Egg Trajectory*: Implemented a trajectory calculation for thrown Spiny Eggs, computing player position, speed, and Lakitu velocity—to rectify an original NES ROM bug where eggs dropped strictly vertically.
  - *Shell Behavior*: Simplified Koopa shell mechanics such that stationary shells never wobble or re-emerge after being stomped.
  - *Lava and Castle Scope*: Designated lava tiles (`v`, `x`) and castle graphics (`A`, `H`) as non-solid backdrops, delegating level completion strictly to author-placed goal triggers (`E`).
- **Deferred Features**: Deferred Cheep Cheep swimming routines (pending water mechanics) and Piranha Plant pipe emergence (pending pipe-attachment design).
- **Legacy Symbol Standardization**: Refactored map loading rules to eliminate older single-letter enemy markers (such as legacy `K` for Koopa) in favor of strict digit-based factory mappings (`0`–`8`) and simplified castle rendering from 21 letter keys down to two multi-cell anchors (`A` and `H`).
- **Testing Fixtures**: Integrated temporary debugging controls—such as forcing immediate game over via the `G` key in `PlayState`—to validate cross-state transitions prior to completing health and death logic.

5 Tools Used

The development of this project relied on a modern suite of software development tools, version control platforms, collaboration channels, and artificial intelligence models. File sharing, communication, and project planning were facilitated through Messenger and Google Drive, while technical documentation and architecture design were maintained through dedicated markdown specifications and diagramming frameworks.

5.1 Environment and Version Control

The development environment and compilation pipeline were built around open-source tools and cross-platform IDEs:

- **Version Control & Repository Management**: GitHub was used as the centralized platform for source code version control, branch management, pull requests, and tracking development issues.
- **Integrated Development Environments**: Visual Studio Code (VS Code) served as the primary code editor, supported by development environment frameworks including AntiGravity and OpenCode for workspace automation.
- **Build System & Toolchain**: CMake managed the cross-platform build configuration (`CMakeLists.txt`), orchestrating automatic source discovery via `file(GLOB_RECURSE ...)` and linking SFML libraries using MinGW/GCC toolchains.

5.2 AI Usage

Explicit Declaration: AI was used in this project.

Generative AI models—including Gemini, DeepSeek, and Claude, were utilized as software design thought partners, code review assistants, and automated refactoring tools throughout development. AI prompting was systematically used to plan complex feature batches, debug physics edge cases, and refine gameplay polish. Representative prompt extracts include:

1. **Batch Feature Planning Prompt**:

   "Ok. Let's plan out a batch fix for this feature:

   - Mario's facing-ness should actually be based on the last player input. That means even when Mario's horizontal velocity is leftwards, if the player last pressed right arrow, Mario should be facing right. This ensures a `visual responsiveness`.
   - Mario's into-the-pipe position and out-of-the-pipe position should actually be shifted 0.5 blocks to the right of what is declared in the `.map` files. This ensures that Mario always interact with the pipes `at the middle`.
   - After Mario gets in the pipe, there should be a brief cut to black, then the scene cuts back to the next area, and Mario gets out of the pipe. This simulates the old SMB vibes.
   - Finally, Castle tiles currently render with the background sky colour too. These pixels should be transparent. Fix this by either doing it with code, or directly manipulating the pixels in the `.png` file."

2. **Rapid-Fire Bug Fix Prompt**:

   "Ok. Let this next batch of tasks be rapid fire round of quick fixes. Plan out the steps for this one.

   - Mario can stand on a wall when he approaches it from the left.
   - The coin bounce should be equal to the death bounce underwater as well.
   - Squished enemies do not play the death bounce animation.
   - Kicked enemies (which are enemies killed by Koopa shells and Mario hitting blocks from below) always play the death bounce animations, but they get flipped upside-down before so.
   - Mario can actually kill the levelup mushroom after consuming a star.
   - Mario's fire projectile should actually move almost thrice as fast as when he's walking normally."

5.3 Media and documentation

Technical design and architectural documentation were authored using structured formats and visual modeling tools:

- **Architecture Diagrams**: Mermaid class and sequence diagrams were created to map out class hierarchies, MVC layer interactions, and the `StateManager` lifecycle.
- **Technical Specifications**: Markdown documents (`.md`) were used to maintain comprehensive technical reports, including the map symbol reference (`MAP_FORMAT.md`), enemy behavior reference (`ENEMIES.md`), and engine core design specs (`CORE_ENGINE_REPORT.md`).
- **Reference Documentation**: System implementation adhered to official SFML 3.0 API specifications, standard C++ OOP guidelines, and technical reference wikis detailing original *Super Mario Bros.* engine mechanics.

6 Features

6.1 Required design features

6.2 Required functionalities

6.3 Advanced functionalities

7 Installation

7.1 GitHub method

7.2 Zip method

8 Usage

Usage tutorial in video form, hosted by *Youtube*:

8.1 Brief tutorial

8.2 Player controls and Character selection

8.3 Power-up states and Transformation mechanics

8.4 Map interactions and Warp pipes

8.5 Enemy behaviors and Combat

9 Conclusion

9.1 Links

9.2 Known issues and limitations

9.3 Future improvements

10 References

- *Design Patterns: Elements of Reusable Object-Oriented Software* by Gamma, Helm, Johnson, and Vlissides
- *SFML* official documentation at https://www.sfml-dev.org/documentation/3.1.0/
- *Super Mario Bros.* Technical Reference and Strategy Wiki
- *Informal source: Super Mario Bros (1985) (Wiki)*
- *Informal source: New Super Mario Bros (Wiki)*
- Course guidelines